- **#undef** : Undefines a previously defined macro.
- **#pragma** : Used for implementation-specific instructions.

9.1.2 Macros

A macro is a fragment of code that gets replaced by a preprocessor before compilation. Macros are defined using the `#define` directive. There are two types of macros in C:

Object-like macros: These are simple replacements. For example:

```
#define PI 3.14159
```

- Whenever the program encounters the identifier `PI` , it is replaced with the value `3.14159` .

Function-like macros: These are more complex and can accept parameters, similar to functions:

```
#define SQUARE(x) ((x) * (x))
```

- In this example, `SQUARE(x)` will be replaced by the expression `((x) * (x))` .

Advantages of Macros:

- Macros allow for code reuse and prevent magic numbers or hardcoded values.
- Function-like macros improve code readability and avoid repeated code.

Disadvantages of Macros: